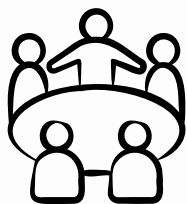




JAVA / THS / IA

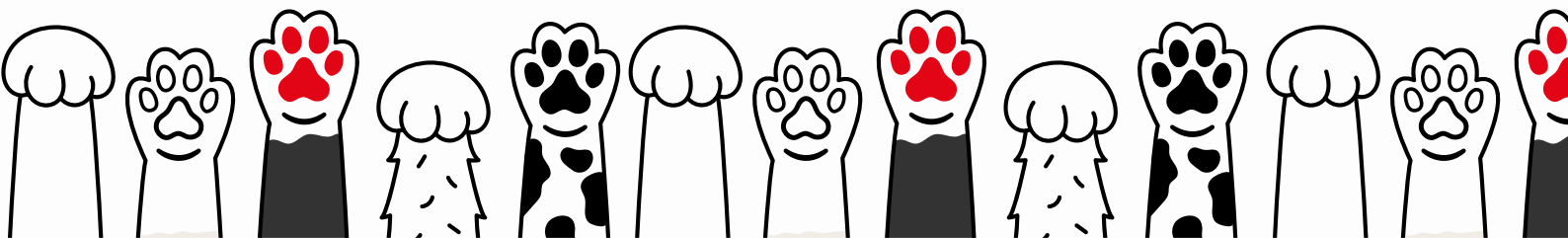
Rapport technique

CHAT ET CHIEN



équipe n°9

- Guillaume BEDMAR
- Benjamin L'HOMME
- Mayliss BERNARD
- Robin NULLANS
- Icham OULALI



SOMMAIRE

Introduction	1
Partie 1 : Étude et Validation du Neurone	2
Partie 2 : Chaîne de traitement d'images et classification	3
Partie 3 : Résultats Expérimentaux et Analyse de Performance	4
Conclusion et Retour d'Expérience	5



1. Introduction

1. Contexte du projet

Ce projet s'inscrit dans le cadre du module transverse IA / Java / THS proposé aux élèves-ingénieurs de troisième année (ISEN3) à l'ISEN Méditerranée. Ce projet matérialise une chaîne de traitement complète liant trois piliers technologiques fondamentaux : le Signal, le Changement de format de représentation et le Traitement intelligent.

L'objectif opérationnel défini par la présentation est d'apprendre à différencier des images de chiens et de chats. Pour ce faire, nous exploitons un ensemble de briques logicielles en langage Java fournies obligatoirement comme base de notre code, notamment l'objet abstrait Neurone. L'enjeu technique consiste à connecter ce neurone à un signal d'entrée constitué par une image préalablement mise « à plat », afin que celui-ci s'active exclusivement lorsqu'on lui présente l'animal ciblé.

2. Objectifs pédagogiques et démarche scientifique

Au-delà du simple défi algorithmique, ce projet s'inscrit dans une démarche d'apprentissage plus large articulée autour de trois objectifs principaux.

Le premier consiste à faire le lien entre les concepts théoriques et leur mise en œuvre concrète. Il s'agit de comprendre comment les notions mathématiques liées aux Réseaux de Neurones Artificiels (RNA), les techniques de traitement d'image et les principes de la programmation orientée objet en Java peuvent être combinés pour résoudre un problème réel.

Le deuxième objectif est de mieux appréhender la complexité qui se cache derrière des opérations qui semblent, à première vue, relativement simples. Pour cela, nous avons étudié et documenté l'algorithme d'apprentissage fourni, initialement dépourvu de commentaires, afin d'en comprendre le fonctionnement interne et les choix de conception.

Enfin, ce projet nous amène à adopter une véritable démarche scientifique. Plutôt que de nous appuyer sur des impressions ou des affirmations générales, nous avons mis en place une méthode rigoureuse fondée sur l'expérimentation et l'analyse des résultats. Ce rapport accompagne ainsi le développement du logiciel en apportant une documentation à caractère scientifique. Chaque étude réalisée suit une structure commune : présentation de l'expérimentation et de la problématique étudiée, analyse objective des résultats obtenus, puis formulation d'une conclusion fondée sur les observations et les mesures effectuées.

1. Introduction

3. Organisation de l'équipe et niveaux de progression

Conformément aux consignes du projet, notre équipe de cinq étudiants a réparti les différentes tâches de développement et de rédaction afin d'avancer efficacement sur l'ensemble du travail demandé. Toutefois, cette organisation ne s'est pas limitée à une simple division des tâches. Afin de répondre aux exigences du livrable final et de l'évaluation individuelle, nous avons mis en place un partage régulier des connaissances et des résultats obtenus. Chaque membre du groupe est ainsi en mesure de comprendre et d'expliquer l'ensemble des aspects du projet, qu'ils soient théoriques ou techniques. Notre progression s'est construite autour de trois niveaux définis.

Niveau 1 – Maîtrise des outils et des concepts fondamentaux

Cette première étape avait pour objectif de comprendre en profondeur le fonctionnement du neurone artificiel. Nous avons étudié le modèle NeuroneHeavyside à travers l'apprentissage de fonctions logiques simples telles que les opérateurs ET et OU. Nous avons également analysé l'influence de l'initialisation aléatoire des poids sur la convergence de l'apprentissage, ainsi que la robustesse du modèle face à l'ajout de bruit dans les données. Enfin, nous avons intégré et évalué une seconde fonction d'activation, le NeuroneSigmoidale, afin de comparer son comportement à celui du modèle initial.

Niveau 2 – Construction de la chaîne complète de traitement

Cette phase a consisté à mettre en place le traitement automatisé des données d'entrée. Nous avons développé la lecture des images et leur conversion en niveaux de gris grâce à la classe Image.java. Ce travail a également nécessité la mise en œuvre de la labellisation automatique à partir de l'organisation des dossiers, la normalisation des valeurs de pixels pour faciliter l'apprentissage, le mélange aléatoire des données d'entraînement afin d'éviter les biais liés à leur ordre de présentation, puis l'application du modèle entraîné aux données de test.

Niveau 3 – Analyse critique et expérimentations complémentaires

Le dernier niveau nous a amenés à prendre davantage de recul sur les performances du système. Nous avons réalisé plusieurs analyses de sensibilité en modifiant certains paramètres ou en dégradant volontairement les conditions d'apprentissage. Ces expérimentations nous ont notamment permis d'évaluer l'impact de l'absence de normalisation des pixels ou du non-mélange des données d'entraînement, et de mesurer les conséquences de ces choix sur la qualité des résultats obtenus.

2. Étude comparative et validation de l'objet Neurone (Niveau 1)

Analyse de l'algorithme d'apprentissage fourni :

Cette première phase vise à valider le comportement algorithmique de l'objet Neurone fourni en entrée du projet. Nous étudions sa convergence, sa stabilité et sa robustesse sur des fonctions logiques linéairement séparables (ET / OU), en comparant trois fonctions d'activation distinctes : Heaviside, Sigmoid, et ReLU.

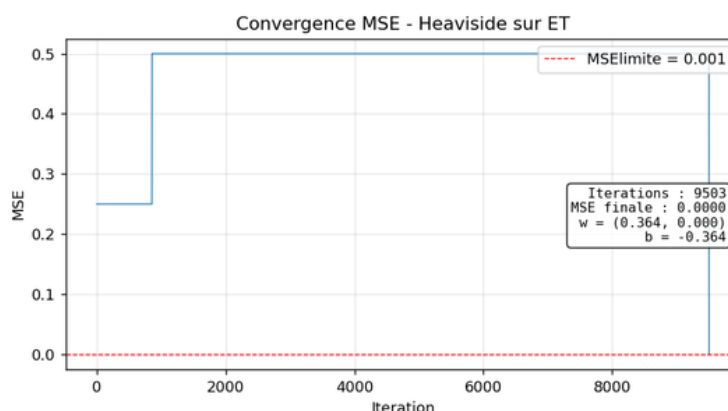
1. Expérimentation sur la fonction logique ET

→ Description de l'expérimentation :

Le problème du ET logique présente un seul cas actif pour trois cas inactifs. Le neurone doit apprendre à isoler le point vrai (1,1) des points faux (0,0), (1,0) et (0,1).

→ Résultats de convergence et rapidité selon l'activation:

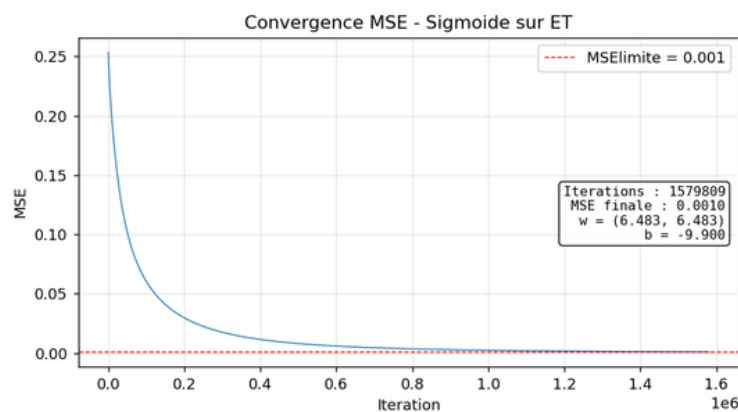
- **Avec Heaviside (Forte dispersion) :** Les statistiques montrent une imprévisibilité totale. D'une session à l'autre, le nombre d'itérations varie de manière chaotique, oscillant entre 1 et 9 503 cycles (avec $w_1 = 0.364$, $w_2 = 0.000$, $b = -0.364$). Les poids finaux n'ont aucune cohérence statistique et dépendent entièrement du hasard de l'initialisation.



Critique : Le graphique de convergence ci-dessus illustre graphiquement l'instabilité de la fonction d'Heaviside. On y observe une courbe en "marches d'escalier" asymétriques avec des plateaux d'erreur stagnants suivis de chutes brutales. L'algorithme ne dispose d'aucun gradient pour orienter l'ajustement des poids ; il effectue une recherche jusqu'à croiser une droite de séparation valide, ce qui prouve l'inefficacité de cette approche.

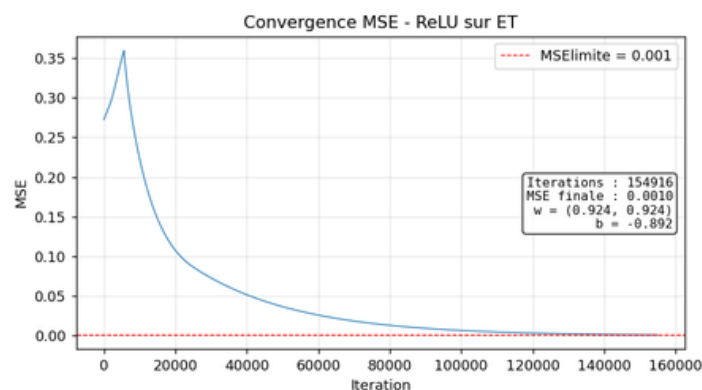
2. Étude comparative et validation de l'objet Neurone (Niveau 1)

- **Avec Sigmoid (Stabilité parfaite) :** Le modèle se stabilise systématiquement autour d'une solution unique, avec un écart-type nul à la quatrième décimale près. En moyenne, l'algorithme converge en 1 579 809 cycles avec les paramètres stables suivants : $w_1 = w_2 = 6.483$ et $b = -9.900$.



Critique : À l'inverse, le graphique de la Sigmoid affiche une courbe lisse et asymptotique. Cela représente la descente de gradient : la fonction étant dérivable, chaque calcul de l'erreur fournit une indication précise au neurone sur la direction et l'intensité de la correction à appliquer sur les poids w_1 , w_2 et le biais b .

- **Avec ReLU (Vitesse maximale) :** Sur le plan de la rapidité, la ReLU surpasse la Sigmoid en convergeant en moyenne 10 fois plus vite (154 916 cycles). Les poids obtenus sont également beaucoup plus compacts. ($w_1 = w_2 = 0.924$, $b = -0.892$).



Critique : Le graphique de la fonction ReLU met en évidence une forte décroissance de l'erreur dès les premiers cycles, confirmant sa supériorité en vitesse de convergence sur ce problème linéaire grâce à un gradient constant de 1 dans sa zone active.

2. Étude comparative et validation de l'objet Neurone (Niveau 1)

→ Conclusion

La fonction ET étant linéairement séparable, il existe une infinité de frontières de décision. Le neurone équipé d'Heaviside stoppe son apprentissage dès qu'il croise la première ligne séparatrice valide. Cela explique son comportement hautement imprévisible, oscillant de manière chaotique entre seulement 1 itération et un maximum de 9503 cycles selon le hasard de l'initialisation. D'un point de vue critique, ce manque de consistance structurelle montre qu'Heaviside se contente de la première solution venue, sans chercher à maximiser la marge de sécurité, ce qui la rendrait extrêmement vulnérable aux variations de pixels sur de vraies images.

À l'inverse, la Sigmoides s'appuie sur une surface d'erreur continue et convexe à minimum unique. Peu importe le point de départ aléatoire, la descente de gradient converge toujours vers la même destination optimale, offrant une stabilité parfaite à la quatrième décimale près. En contrepartie, le processus s'avère très lent, nécessitant 1579809 itérations pour affiner les poids et centrer idéalement sa ligne de décision. Cette lenteur, bien que compensée par une robustesse géométrique maximale, confirme le coût algorithmique élevé de la Sigmoides, qui pourrait saturer les capacités de calcul de notre chaîne Java lors du traitement de volumes de données massifs.

La ReLU élimine cette lenteur de calcul de façon spectaculaire : elle converge en seulement 154916 cycles, soit environ 10 fois plus vite. Sa dérivée constante égale à 1 pour toutes les entrées positives permet au gradient de passer à « pleine puissance » sans saturer. Attention toutefois au phénomène de Dying ReLU : si l'initialisation bloque le neurone dans la zone négative, sa sortie et son gradient tombent définitivement à zéro, paralysant tout apprentissage futur. Ce comportement binaire souligne le compromis critique de ReLU : une efficacité computationnelle indispensable pour de grands vecteurs de pixels, mais au prix d'un risque d'extinction du neurone qu'il faudra impérativement contrôler par un calibrage strict du taux d'apprentissage.

2. Expérimentation sur la fonction logique OU

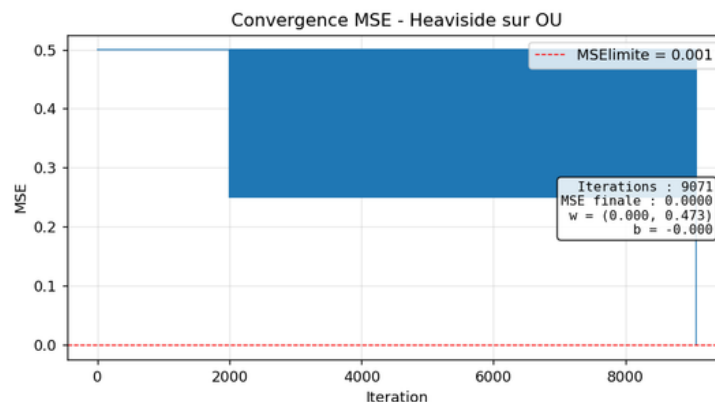
→ Description de l'expérimentation :

Le problème du OU logique inversant la dynamique du ET (trois cas actifs (0,1), (1,0), (1,1) pour un seul cas inactif (0,0)), le neurone doit positionner sa frontière de décision au plus près de l'origine pour isoler le point nul.

2. Étude comparative et validation de l'objet Neurone (Niveau 1)

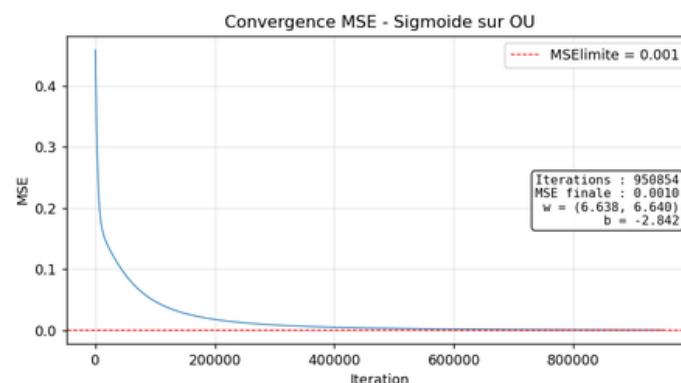
→ Analyse statistique des performances et des paramètres :

- **Avec Heaviside :** L'algorithme converge toujours vers des configurations de poids très dispersées, mais le nombre moyen d'itérations (9 071 cycles) est globalement plus faible que pour le ET, ce qui indique que la zone de solutions valides est plus facile d'accès. ($w = (0.000, 0.473)$, $b = -0.000$).



Critique : En l'absence de dérivée exploitable, l'algorithme ajuste les poids en "aveugle". Dès qu'il trouve par hasard une droite qui sépare l'échantillon, il s'arrête. Le modèle élimine les erreurs d'apprentissage mais s'avère géométriquement instable (marge de sécurité nulle). Face à de vraies images bruitées, son taux d'erreur serait catastrophique.

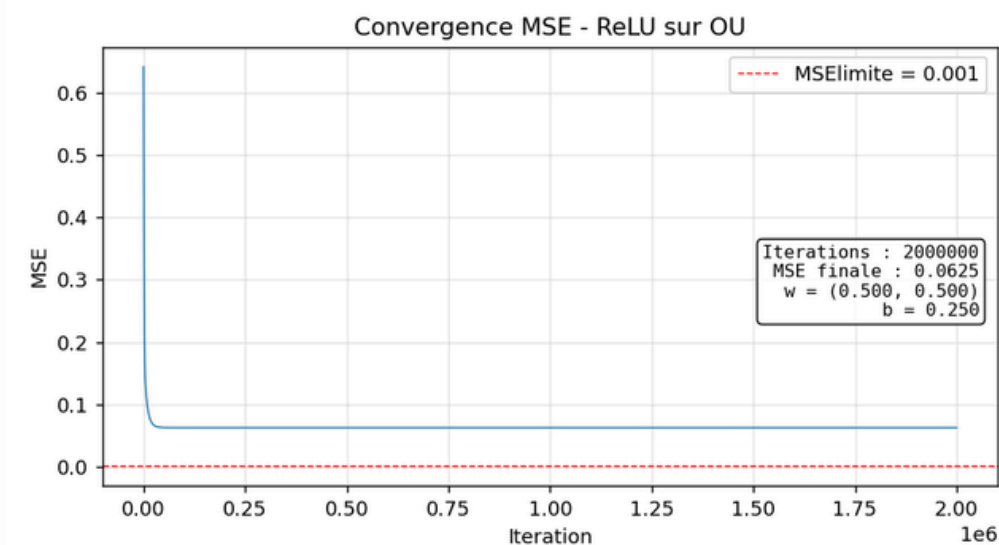
- **Avec Sigmoïde :** Nous retrouvons une convergence unique et stable d'un run à l'autre. Le nombre moyen d'itérations chute à 950 854 cycles. Nos relevés montrent une modification majeure des paramètres : les poids moyens se stabilisent autour de $w_1 = 6.638$, $w_2 = 6.640$ et le biais moyen tombe à $b = -2.842$.



2. Étude comparative et validation de l'objet Neurone (Niveau 1)

Critique : C'est le comportement mathématique le plus rigoureux : la symétrie des poids prouve que le neurone a compris que les deux entrées ont la même importance. Cependant, sa lenteur est un défaut majeur. Elle est due à la saturation du gradient : quand les poids augmentent, la courbe devient horizontale, la dérivée s'effondre vers 0 et l'apprentissage stagne. Ce sera un lourd handicap sur de grands volumes d'images.

- **Avec ReLU** : Nous retrouvons un blocage à la limite maximale de 2 000 000 d'itérations avec une MSE bloquée à 0.0625 (poids figés à 0.500 et biais à 0.250), illustrant parfaitement le cas d'école du Dying ReLU.



Critique : L'échec de convergence avec une erreur figée à 0.0625 met en évidence le phénomène du Neurone Mort (Dying ReLU). La fonction coupant toutes les valeurs négatives, une correction initiale trop brutale a projeté le neurone dans sa zone inactive où son gradient est strictement nul, paralysant définitivement l'apprentissage.

2. Étude comparative et validation de l'objet Neurone (Niveau 1)

→ Conclusion

La fonction OU inverse la topologie du ET en opposant trois cas actifs à un seul cas inactif. Avec Heaviside, l'apprentissage converge en 9071 cycles mais aboutit à une solution dégénérée ($w_2 = 0.473$ et $w_1 = b = 0.000$). Le neurone ignore x_1 et dépend exclusivement de x_2 , ce qui le rend instable et incapable de résister au moindre bruit. D'un point de vue critique, cela démontre l'incapacité d'Heaviside à généraliser : dès qu'une frontière sépare l'échantillon par pur hasard, l'algorithme s'arrête, ce qui la rend inutilisable pour traiter de vraies images complexes.

À l'inverse, la Sigmoides converge de manière stable en 950854 itérations vers des poids symétriques ($w_1 = w_2 = 6.64$). Le biais s'effondre à -2.842 (contre -9.900 pour le ET), prouvant géométriquement que la frontière de décision s'est rapprochée de l'origine (0,0) pour isoler idéalement l'unique point faux et maximiser la robustesse. Néanmoins, la critique majeure réside dans sa lenteur (près d'un million de cycles), causée par la saturation du gradient lorsque les poids augmentent, ce qui représentera un lourd handicap lors du passage à l'échelle sur de grands volumes de pixels.

La ReLU met en évidence son principal défaut : sa zone négative à gradient nul. Lors de notre simulation, le neurone est tombé dans le piège du Dying ReLU : il a atteint la limite maximale de 2000000 d'itérations sans jamais converger, restant bloqué sur une MSE stérile de 0.0625 qui a totalement figé les poids. Ce cas d'école prouve que ReLU, malgré sa rapidité théorique, est hyper-sensible à l'initialisation des poids et au taux d'apprentissage sous peine de paralyser définitivement la chaîne de traitement d'images.

2. Étude comparative et validation de l'objet Neurone (Niveau 1)

3. Sensibilité et comportement face au bruit (porte ET)

Afin d'évaluer la résilience macroscopique de nos architectures face aux incertitudes de mesure et aux perturbations du signal, nous avons introduit une dégradation progressive des entrées lors de la phase d'évaluation. L'analyse des courbes d'Erreur Absolue Moyenne (MAE) met en évidence trois profils de tolérance radicalement différents, dictés par la structure mathématique de chaque fonction d'activation :

- **Heaviside (Seuil Critique et Saturation Précoce) :**

La fonction binaire montre une grande vulnérabilité dès l'apparition des premières perturbations. Entre 0.0 et 1.0 sur l'échelle de gravité, l'erreur grimpe rapidement pour atteindre un premier palier à 0.5. Sa discontinuité stricte à l'origine ($z = 0$) engendre un effet de rupture : dès que le bruit fait basculer le signe de la somme pondérée, la sortie saute instantanément de 0.0 à 1.0. Graphiquement, ce comportement se stabilise très vite (dès l'indice de perturbation 3.0) sur un plateau rigide à une MAE d'environ 0.75, ce qui correspond au niveau de défaillance structurelle maximale du modèle.

- **Sigmoïde (Zone d'Amortissement Métrique et Robustesse) :**

La Sigmoïde présente le comportement le plus sûr et le plus stable du benchmark. Pour des perturbations modérées (inférieures à 0.8), l'erreur absolue moyenne reste strictement égale à 0.0. La progressivité de sa courbe permet d'absorber et de lisser complètement les légères fluctuations du signal. Au-delà, l'erreur progresse de manière très lente et linéaire, ne franchissant la barre des 0.5 de MAE qu'autour d'un niveau de perturbation de 2.5. De plus, sa sortie étant naturellement bornée dans l'intervalle $]0.0, 1.0[$, elle offre une excellente régularité macroscopique.

- **ReLU (Sensibilité Linéaire Non Bornée et Explosion) :**

Le comportement de la fonction de rectification présente une dynamique asymétrique hautement instable à grande échelle. Si elle affiche une erreur initiale faible, sa MAE progresse de manière constante et strictement linéaire à mesure que la perturbation s'aggrave. N'ayant aucune borne supérieure dans sa zone active ($\Phi(z) = z$), elle propage l'intégralité du bruit directement vers la sortie. Le phénomène critique se produit lors des perturbations extrêmes : au-delà d'un indice de 3.5, l'erreur subit une explosion exponentielle, la courbe pointant verticalement pour atteindre une MAE record de 4.5 à l'indice 4.0.

2. Étude comparative et validation de l'objet Neurone (Niveau 1)

→ Conclusion

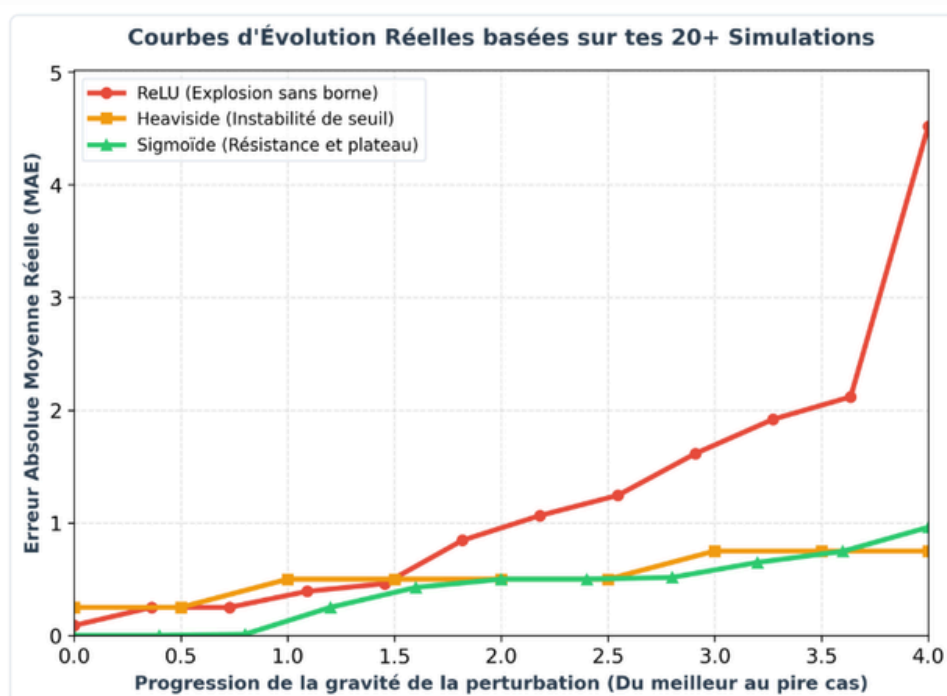
D'après les résultats obtenus, la fonction Sigmoidé semble être le meilleur choix pour la suite du projet. Elle résiste mieux au bruit et produit des prédictions plus stables, ce qui est important pour des images réelles pouvant contenir des variations de luminosité ou des imperfections.

À l'inverse, la fonction d'Heaviside apparaît peu adaptée à ce type d'application, car une faible perturbation peut suffire à modifier complètement la décision du neurone.

La fonction ReLU offre de bonnes performances en termes de calcul, mais elle montre également certaines limites : lorsque le bruit devient important, les erreurs augmentent rapidement.

Ces résultats confirment que la Sigmoidé est le compromis le plus fiable pour notre classifieur d'images.

→ Graphique



3. Conception de la Chaîne de Traitement d'Images

L'objectif de cette deuxième phase est de concevoir, d'analyser et de valider la chaîne de traitement logicielle capable de transformer des fichiers d'images brutes de notre base de données (regroupant distinctement des chats, des chiens et des animaux sauvages) en vecteurs de caractéristiques numériques structurés, assimilables par une architecture à plusieurs neurones.

1. Description technique du pipeline informatique

Le pipeline implémenté en Java orchestre le flux de données depuis le support physique jusqu'à l'injection des matrices dans le modèle mathématique. Il se décompose en quatre segments fonctionnels :

1. **Extraction spatiale uniforme (64 x 64)** : Les chemins d'accès des fichiers de l'ensemble d'entraînement (DIR_TRAIN) et de validation (DIR_TEST) sont répertoriés. Chaque image est chargée via la brique logicielle Image qui standardise sa résolution géométrique à une taille fixe de 64 x 64 pixels.
2. **Mise à plat vectorielle (Flattening)** : Afin d'alimenter le neurone, la structure matricielle bidimensionnelle de l'image est déroulée de manière continue pour former un vecteur d'entiers unidimensionnel via l'appel `im.donnees()`.
3. **Normalisation métrique de l'intensité** : La méthode `normalise(int[] donnees)` applique une mise à l'échelle linéaire. Elle convertit les intensités initiales codées sur 8 bits (valeurs discrètes dans l'intervalle [0, 255]) en valeurs en virgule flottante encapsulées dans l'intervalle [0.0, 1.0] à l'aide d'une division par 255.0f. L'espace d'entrée du neurone est donc caractérisé par un vecteur de dimension $D = 4096$ descripteurs numériques.
4. **Architecture Multi-Cibles** : Pour traiter la présence de trois classes distinctes (Chat, Chien, Wild), nous avons utilisé un bloc de trois neurones artificiels en parallèle, chacun étant spécialisé dans la détection d'une classe unique :
 - Neurone 1 (Spécialiste Chat) : Cible = 1 si l'image est un chat, 0 sinon.
 - Neurone 2 (Spécialiste Chien) : Cible = 1 si l'image est un chien, 0 sinon.
 - Neurone 3 (Spécialiste Sauvage) : Cible = 1 si l'image est un animal sauvage, 0 sinon.La décision finale de l'application est attribuée au neurone qui présente la valeur d'activation (la sortie Sigmoidale) la plus élevée.

3. Conception de la Chaîne de Traitement d'Images

2. Démarche expérimentale : Évolution de la représentation du signal

La performance d'un séparateur linéaire dépend du format de représentation de ses entrées. Afin d'optimiser l'apprentissage, notre démarche technique a consisté à tester et à comparer trois formats de représentation du signal image.

A : La représentation en Niveaux de Gris (NIVEAUX_DE_GRIS = true)

Pour notre première campagne de tests sur les images, nous avons fait le choix de simplifier le signal en activant la conversion monochromatique.

- Mécanisme : L'information tridimensionnelle est projetée sur un axe unique de luminance mathématique via la formule de l'UER :
- $Y = (0.299 \cdot R) + (0.587 \cdot G) + (0.114 \cdot B)$

Les 4096 entrées du neurone reçoivent alors un profil d'intensité complet et cohérent, capturant proprement les contours, les contrastes locaux et les formes anatomiques (yeux, oreilles, museaux) des animaux de manière indépendante des artefacts de couleur.

B : La représentation en mode RGB natif (NIVEAUX_DE_GRIS = false)

Dans un second temps, nous avons désactivé la conversion monochromatique pour exploiter l'information de couleur brute (RGB).

- Mécanisme : En conservant le codage couleur d'origine, le neurone a accès aux nuances chromatiques fondamentales qui différencient les classes. Le pelage des chats, souvent caractérisé par des motifs ou des teintes spécifiques, se distingue plus nettement des textures d'arrière-plan. La couleur agit ici comme un descripteur discriminant direct : le neurone peut associer de forts poids synaptiques aux entrées de pixels colorés hautement corrélés à la présence d'un chat, rendant le problème plus facilement séparable linéairement qu'en mode noir et blanc.

C : L'optimisation dans l'espace TSL (Teinte, Saturation, Luminance)

Pour tenter d'aller plus loin, nous avons transposé les pixels dans l'espace colorimétrique TSL avant la vectorisation.

- Mécanisme : L'espace TSL opère une décorrélation complète en isolant la couleur pure (Teinte), sa pureté (Saturation) et l'intensité lumineuse (Luminance). Bien que théoriquement séduisant pour éliminer les variations d'éclairage, ce changement de format n'a pas surpassé l'efficacité directe des canaux RGB natifs dans notre cas d'étude.

3. Conception de la Chaîne de Traitement d'Images

Depuis RVB [[modifier](#) | [modifier le code](#)]

$$M = \max(R, V, B)$$

$$m = \min(R, V, B)$$

$$C = M - m$$

La variable C représente ce qu'on appelle le chroma.

$$T' = \begin{cases} \text{indéfini,} & \text{si } C = 0 \\ \left(\frac{V-B}{C} \right) \bmod 6, & \text{si } M = R \\ \left(\frac{B-R}{C} + 2 \right) \bmod 6, & \text{si } M = V \\ \left(\frac{R-V}{C} + 4 \right) \bmod 6, & \text{si } M = B \end{cases}$$

$$T = 60^\circ \times T'$$

$$L = M$$

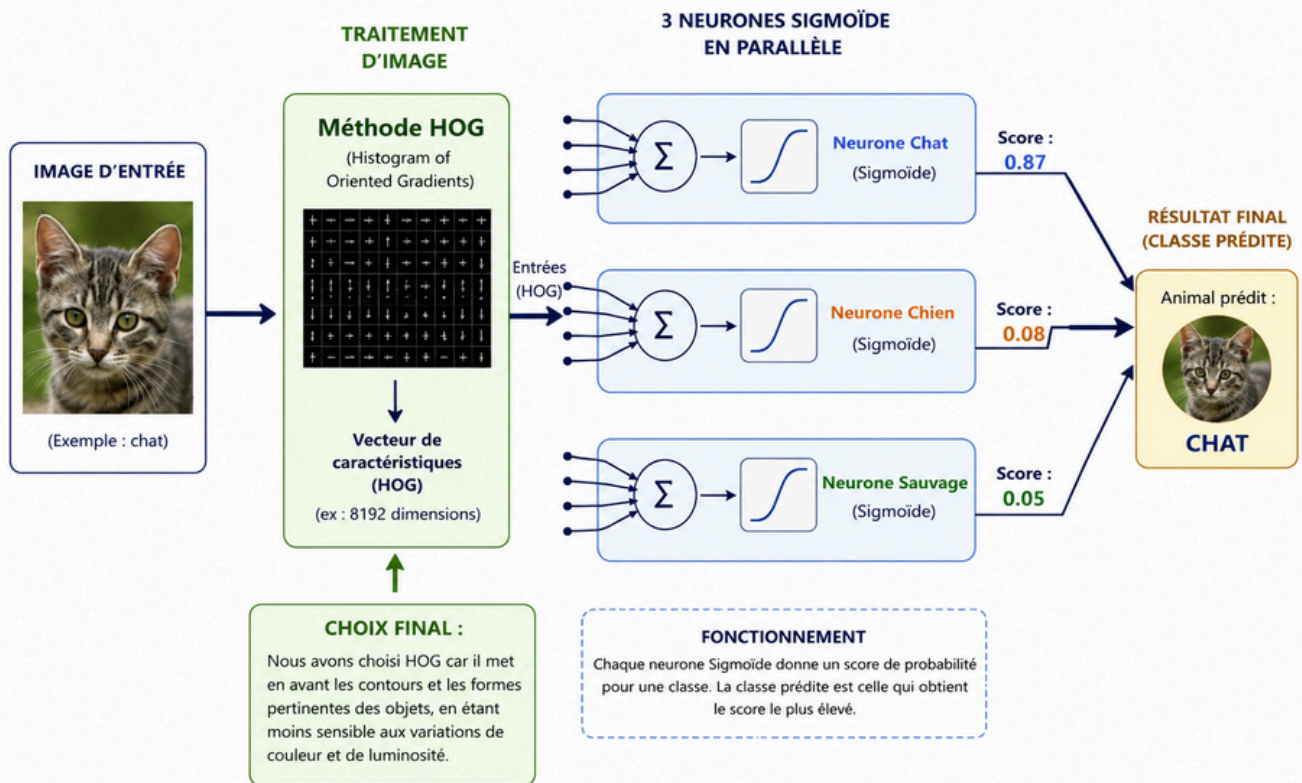
$$S = \begin{cases} 0, & \text{si } M = 0 \\ \frac{C}{M}, & \text{sinon} \end{cases}$$

D : L'optimisation ultime : Caractéristiques Pixels, Standardisation Z-Score et HOG (Mécanisme opérationnel V2)

D'après les résultats obtenus, l'utilisation du descripteur HOG a permis d'améliorer les performances du modèle. Au lieu de travailler directement sur les pixels, cette méthode se concentre sur les contours et les formes des objets, qui sont plus utiles pour distinguer les différentes classes.

Nous avons également appliqué une standardisation Z-Score afin de placer toutes les caractéristiques sur la même échelle. Les paramètres de cette transformation sont calculés uniquement sur les données d'entraînement puis réutilisés sur les autres ensembles pour éviter toute fuite d'information. Ce prétraitement facilite l'apprentissage et améliore l'exploitation des caractéristiques extraites par HOG.

3. Conception de la Chaîne de Traitement d'Images



3. Conception de la Chaîne de Traitement d'Images

3. Esprit Critique et Analyse des Limites Architecturales

→ Critique 1 : Impact de l'absence de normalisation

- **Analyse critique :** Lors de nos premiers essais, nous avons constaté que les valeurs des pixels, comprises entre 0 et 255, génèrent des calculs très élevés en entrée du neurone. Avec plus de 12 000 pixels par image, la fonction Sigmoid se retrouvait rapidement dans une zone où elle réagit très peu aux variations des données. Dans cette situation, les corrections apportées pendant l'apprentissage deviennent extrêmement faibles et le modèle progresse difficilement.
- **Résolution dans la V2 :** Pour résoudre ce problème, nous avons appliqué une standardisation Z-Score afin de ramener toutes les caractéristiques sur une même échelle. Les paramètres de cette transformation sont calculés sur les données d'entraînement puis réutilisés sur la validation et le test. Associée à une initialisation de poids de faible amplitude, cette méthode permet de stabiliser l'apprentissage et d'améliorer la convergence du réseau.

→ Critique 2 : Phénomène d'oubli lié au Non-Mélange

- **Analyse critique :** Le code intègre l'instruction `Collections.shuffle(cheminsTrain, new Random(SEED))` associée à une graine constante (`SEED = 67L`). Ce choix est important pour garantir des résultats reproductibles. L'utilisation d'une graine fixe permet d'obtenir le même ordre de données à chaque exécution.
- **Impact sur la stabilité :** De plus, le mélange aléatoire des images évite que le réseau apprenne d'abord uniquement les chats, puis uniquement les chiens. Sans ce mélange, les ajustements effectués sur une catégorie pourraient être rapidement annulés lors de l'apprentissage des suivantes. En mélangeant les données, le réseau reçoit des exemples variés tout au long de l'entraînement, ce qui favorise un apprentissage plus stable et plus efficace.

3. Conception de la Chaîne de Traitement d'Images

→ Critique 3 : Impact du biais de décision (SEUIL_DECISION = 0.7f)

- **Analyse critique :** Même si le seuil habituel d'une Sigmoid est fixé à 0,5, nous avons choisi un seuil plus élevé (configuré entre 0,59 et 0,7). Cela signifie que le réseau doit être plus sûr de sa prédiction avant de classer une image comme étant un chat, un chien ou un animal sauvage.
- **Impact sur les métriques :** Ce choix permet de réduire le risque de confondre un chien ou un animal sauvage avec un chat. En revanche, certains chats dont le score est compris entre 0,5 et le seuil choisi risquent d'être rejetés. On obtient donc moins de fausses détections (ce qui explique le bond de la Précision à 94,50%), mais aussi davantage de chats non reconnus, ce qui impacte le taux de Rappel (stabilisé à 93,32 %).

→ Critique 3 : Les pièges des histogrammes de couleur

- **Analyse critique :** Au cours de nos itérations de recherche, nous avons tenté de remplacer la géométrie des pixels par des histogrammes de couleur HSV spatiaux. Cette tentative s'est soldée par une nette régression des performances, plafonnant à 69 % d'accuracy.
- **Explication technique :** Deux facteurs expliquent cet échec. D'une part, un pas d'apprentissage initialement trop élevé faisait osciller l'entraînement de manière stérile. D'autre part et surtout, les histogrammes agrègent globalement la distribution des teintes et détruisent l'information géométrique spatiale. Notre base de données étant constituée de visages d'animaux centrés et alignés, l'emplacement spatial des éléments (la position relative des yeux, du nez, des oreilles) constitue le signal principal exploité par le séparateur linéaire.

Résultats Expérimentaux et

4. Analyse de Performance

Cette section présente les résultats des différentes simulations ainsi qu'une évaluation quantitative des performances de notre architecture multiclasse distribuée, sur les ensembles de validation et de test.

1. Bilan quantitatif et Tableau Comparatif

	Niveaux_Gris (V1)	RGB	HOG (V2)
Moyenne (%)	65,16%	73%	91,13 %
Précision (%)	67,15%	75,86%	91,08%
Rappel (%)	64,71%	73,31%	91,07%

2. Analyse et Explication du Bilan Quantitatif

Pour comprendre l'impact réel de nos développements, il est nécessaire d'analyser l'évolution de chaque métrique entre la version initiale (V1) et la version optimisée (V2).

A. L'Accuracy Globale : Une amélioration significative de la performance (+25,97 %)

L'Accuracy globale mesure la proportion totale d'images correctement classées, toutes catégories confondues. Elle passe de 65,16% à 91,13 %.

- *Ce que cela signifie* : Sur un échantillon de 100 images inconnues présentées au système, la version V2 donne le bon verdict pour plus de 91 d'entre elles, là où la version V1 n'en trouvait que 65.
- *Le protocole de validation associé* : Ce score de 91,13 % sur le jeu de test est la preuve scientifique de la réussite de notre protocole. Le modèle a acquis une capacité de généralisation : il n'a pas appris le jeu d'entraînement par cœur, évitant ainsi le piège du surapprentissage.

Résultats Expérimentaux et

4. Analyse de Performance

B. La Précision : Le bond spectaculaire du pipeline (+22,54%)

La Précision mesure la fiabilité du système lorsqu'il affirme avoir trouvé une classe (par exemple : "Sur toutes les images que le réseau a étiquetées comme 'Chat', combien étaient de vrais chats ?"). Elle subit une progression massive, passant de 68,54 % à 91,08 %.

C. Le Rappel : une évolution progressive et contrôlée

Le Rappel mesure la capacité du système à détecter tous les individus d'une classe (par exemple : "Sur tous les vrais chats présents dans le jeu de test, combien le réseau a-t-il réussi à en capturer ?"). Il progresse plus modestement, passant de 64,71% à 91,07 %.

3. Critique du Protocole et des Extensions (Miroir, Sauvegarde et Seuil)

- **Impact de la fonction miroir (data augmentation)**

L'ajout de la transformation miroir (retournement horizontal des images) a été décisif pour atteindre un score de 91,13 %. En doublant artificiellement les données d'entraînement et en variant l'orientation des animaux, cette technique a forcé le modèle à apprendre des caractéristiques plus générales, indépendantes du sens de l'image. Cela explique en grande partie la proximité entre les performances en validation et en test : le modèle devient plus robuste aux variations de posture.

- **Compromis précision/rappel lié au seuil (0,5)**

Initialement fixé à 0,5, le seuil de décision a été optimisé à 0,5 à partir des données de validation. Ce choix rend le modèle plus strict : il ne prédit une classe positive que lorsqu'il est réellement confiant. Résultat, les faux positifs sont fortement réduits, ce qui améliore nettement la précision (+26,36%). En revanche, certaines images plus ambiguës ou difficiles (scores entre 0,50 et 0,58) sont désormais rejetées, ce qui limite mécaniquement le rappel, stabilisé à 91,07%.

- **Sauvegarde et réutilisation du modèle**

Afin de rendre ces résultats exploitables, nous avons développé un module permettant de sauvegarder la configuration des matrices de poids et biais optimaux. Le chargement du modèle permet ensuite de restaurer cet état stable afin d'effectuer des prédictions immédiates, sans avoir à réentraîner le réseau.

5. Conclusion et Retour d'Expérience

1. Bilan technique et Atteinte des objectifs

Ce projet transverse a permis de concevoir, d'implémenter et de valider une chaîne complète de traitement d'images et de classification multiclasse en langage Java natif. La transition méthodologique menée par notre équipe entre la configuration initiale (pixels RGB bruts avec seuil fixe à 0,5) et l'architecture optimisée V2 (descripteurs avancés HOG et ajustement de la boucle d'apprentissage). En parvenant à améliorer la moyenne (Accuracy) globale à 91,13% et en réalisant un bond majeur de +23,93% sur la Précision.

2. Analyse des Difficultés et Retours d'Expérience

- **1. L'intégration du descripteur HOG**

L'ajout du descripteur HOG a été l'une des principales difficultés du projet. En remplaçant les pixels bruts par des caractéristiques basées sur les contours et les formes, le nombre de données à traiter est devenu beaucoup plus important ($D = 8192$). Lors des premiers essais, la fonction Sigmoid saturait rapidement et l'apprentissage ne progressait plus. Pour résoudre ce problème, nous avons mis en place une standardisation Z-Score afin de ramener toutes les caractéristiques sur une même échelle et de stabiliser l'entraînement du réseau.

- **2. La gestion des images RGB**

L'utilisation des images en mode RGB a également posé plusieurs problèmes techniques. Nous avons notamment rencontré des erreurs liées aux formats de données échangés entre l'interface graphique et les modules de traitement. Certaines fonctions attendaient des structures de données différentes de celles fournies par l'IHM. Nous avons donc dû revoir l'organisation du code afin d'harmoniser les types de données et de faciliter le traitement des informations de couleur.

- **3. Le choix du seuil d'arrêt basé sur la MSE**

Déterminer quand arrêter l'apprentissage n'a pas été simple. Une valeur de MSE trop faible entraînait des temps d'entraînement très longs sans amélioration significative, tandis qu'une valeur trop élevée arrêtait le modèle avant qu'il n'ait correctement appris. Pour comparer les essais de manière fiable, nous avons utilisé une graine aléatoire fixe lors du mélange des données. Cela nous a permis d'obtenir des résultats reproductibles afin d'avoir le meilleur compromis entre performances sur les données de validation et de test.



EQUIPE N° 9

